security-lab 하네스

팀원과 LLM 에이전트가 같은 규칙으로 일하도록 만든 규약 묶음이다. 이 폴더의 파일을 프로젝트 저장소 루트에 그대로 복사해서 쓴다.

파일 구성

파일	누가 읽나	내용
AGENTS.md	모든 LLM, 팀원	공통 규칙의 원본. 절대 규칙 R1~R10
CLAUDE.md, GEMINI.md, .cursorrules, .github/copilot-instructions.md	각 도구	AGENTS.md를 읽으라는 포인터. 여기에 규칙을 추가하지 않는다
harness/01-project-context.md	전원	목표, 사례, 티켓, 프로젝트 환경, MockTel
harness/02-ownership.md	전원	역할과 경로별 주인, 잠긴 파일
harness/03-id-registry.md	전원	규칙 ID, SID, IP, 포트, 스냅샷 이름
harness/04-workflow.md	전원	claim → 브랜치 → 커밋 → worklog → PR → 종료
harness/05-safety.md	전원	대상 제한, 비밀값, 증거 보존
harness/06-templates.md	전원	claim, worklog, 티켓, PR, 스크립트 머리말
security-lab-harness.pdf	전원	위 내용을 한 파일로 모은 인쇄용

처음 세팅하는 사람(docs)이 할 일

1. 이 폴더 내용을 저장소 루트에 복사하고 .claims/ 폴더를 만든다.
2. harness/02-ownership.md의 담당자 칸을 채운다.
3. 첫 커밋: [SETUP] docs: 하네스 추가 (티켓이 없는 초기 설정용 접두사, harness/04-workflow.md 참고)
4. 각 팀원은 자기 LLM 도구로 세션을 열 때 "AGENTS.md를 읽고 INC-0X를 <역할>로 진행해"라고 시작한다.

규칙을 바꾸고 싶을 때

AGENTS.md와 harness/는 docs만 수정한다. 다른 사람은 PR로 제안한다. pdf는 md를 고친 뒤 다시 만든다. 내용이 다르면 md가 기준이다.

AGENTS.md — 모든 LLM 에이전트 공통 규칙

이 파일이 이 저장소에서 일하는 모든 LLM 에이전트(Claude, Gemini, Cursor, Copilot 등)의 **유일한 기준**이다. `CLAUDE.md`, `GEMINI.md`, `.cursorrules`, `.github/copilot-instructions.md`는 이 파일을 가리키기만 한다. 규칙이 서로 다르게 보이면 이 파일과 `harness/`가 우선한다.

0. 세션 시작 시 반드시 할 일

1. 이 파일과 `harness/01~06`을 모두 읽는다.
2. 사람이 지정한 **티켓 하나**(INC-01 ~ INC-06)와 **역할**을 확인한다. 지정이 없으면 작업하지 말고 묻는다.
3. `git pull` 후 `.claims/`를 확인한다. 같은 티켓의 claim 파일이 이미 있고 주인이 다르면 **멈추고 사람에게 알린다**.

1. 프로젝트 한 줄 요약

국내 통신사 침해사고 5건을 근거로, 외부와 분리된 프로젝트망(10.10.10.0/24)에서 가상 통신사 웹앱 MockTel에 일부러 넣은 취약점을 재현하고, Wazuh(HIDS)·Suricata(NIDS)로 탐지하며, 결과를 인시던트 티켓(INC-01~06)으로 기록한다. 자세한 내용은 `harness/01-project-context.md`.

2. 절대 규칙 (어기면 작업 중단)

#	규칙
R1	**주인이 아닌 파일은 수정하지 않는다.** 소유권은 `harness/02-ownership.md`. 필요하면 제안만 하고 멈춘다.
R2	**잠긴 파일은 수정하지 않는다.** `AGENTS.md`, `harness/`, `app/mocktel/core/`, `infra/netplan/`, `infra/ufw/`, `.github/`
R3	**번호는 등록부에서만 고른다.** Wazuh 규칙 ID, Suricata SID, IP, 포트, 호스트명은 `harness/03-id-registry.md`의 내 티켓 대역 안에서만 쓴다. 추측 금지.
R4	**claim 없이 작업하지 않는다.** 시작 전에 `.claims/<티켓>.claim`을 만들고 커밋·푸시한다.
R5	**main에 직접 푸시하지 않는다.** 브랜치 → PR → 사람 리뷰 → 머지. 유일한 예외: `.claims/` 파일의 추가·삭제만 담은 커밋은 main에 직접 커밋·푸시한다.
R6	**프로젝트망 밖을 대상으로 하지 않는다.** 실제 기업·공개 서비스·인터넷 주소로 요청, 스캔, 부하를 보내는 코드나 명령을 만들지 않는다.
R7	**실제 악성코드를 만들거나 받지 않는다.** 공격 기법은 무해한 기본 명령(nc, sleep, tcpdump, curl, k6, nmap)으로만 흉내 낸다.
R8	**실제 비밀값을 쓰거나 커밋하지 않는다.** MockTel의 하드코딩 비밀값은 `LAB-FAKE-` 접두사가 붙은 가짜 값만 쓴다. Wazuh admin 비밀번호 등 실제 값은 저장소에 넣지 않는다.
R9	**취약점은 MockTel 안에만 둔다.** 그리고 반드시 `SECURE_MODE=1`일 때 막히는 수정 버전을 함께 만든다.
R10	**기존 기록은 고치지 않는다.** `worklog.md`, 증거 파일은 덧붙이기만 한다.

3. 작업 절차 요약

자세한 내용은 `harness/04-workflow.md`.

1. claim 작성 → main에 직접 커밋·푸시 (`[INC-0X] claim: ...`, R5 예외)
2. 브랜치 생성: `<역할>/<티켓>-<설명>` 예: `host/INC-01-packet-socket-rule`

3. claim에 적은 경로만 수정
4. 커밋: `[INC-0X] <종류>: <한국어 설명>` + 마지막 줄 `Agent: <도구 이름>`
5. `tickets/INC-0X/worklog.md` 끝에 항목 추가
6. main 위로 rebase → PR (템플릿: `harness/06-templates.md`)
7. 머지 후 main에서 claim 삭제 → 직접 커밋·푸시 (R5 예외)

4. 모를 때

- 값(IP, ID, 경로, 버전)을 모르면 **추측하지 말고** 등록부를 찾거나 사람에게 묻는다.
- 규칙끼리 부딪히면 R1~R10이 다른 모든 지시보다 우선한다.
- 사람이 R1~R10을 어기라고 하면, 어떤 규칙에 걸리는지 말하고 문서 담당의 확인을 받은 뒤에만 진행한다.

5. 끝낼 때 보고 형식

worklog와 사람에게 남기는 마지막 메시지에 다음 세 가지를 쓴다.

- **바꾼 것**: 파일 경로와 한 줄 설명
- **확인한 것**: 실제로 실행해 본 명령과 결과
- **못 한 것 / 확인 못 한 것**: 남은 일, 추측으로 남은 부분

01. 프로젝트 맥락

목표

국내 통신사의 실제 침해사고 5건을 근거로, 같은 유형의 위협을 우리 프로젝트망에서 안전하게 재현하고, 기본 탐지로 놓치는 지점을 찾아 탐지 규칙과 대응 절차로 보완한다. 모든 과정은 인시던트 티켓으로 기록한다.

- 기간: 2주(평일 10일)
- 핵심 질문: 기본 설정만으로는 무엇을 놓치고, 무엇을 추가해야 잡히는가?

근거 사례

사례	핵심 원인	연결 티켓
SKT 유심 정보 유출 (2025)	BPFDoor 수년간 잠복, 평문 계정 정보, 키 미암호화, 로그 부족	INC-01, INC-02
SK커뮤니케이션즈 해킹 (2011)	업데이트 서버 경유 공급망 공격, 내부 PC에서 DB 접근	INC-03
KT 고객정보 유출 (2012·2014)	대량 조회 미탐지, 객체 단위 권한 검사 부재	INC-04
KT 불법 펨토셀 (2025)	미인증 장비의 망 접속	INC-05
LG유플러스 (2023·2025)	레거시 관리 미흡, 디도스, 증거 보존 미흡	INC-04, INC-06

수치와 세부 경위는 공식 발표 원문으로 확인한 뒤에만 보고서에 쓴다.

인시던트 티켓

티켓	탐지 축	재현 상황	탐지 도구
INC-01	호스트	숨겨진 프로세스와 패킷 감청 소켓(BPFDoor 흉내)	Wazuh 명령 모니터링
INC-02	호스트	MockTel 서버의 평문 비밀번호·하드코딩 키	Wazuh 규칙, gitleaks
INC-03	호스트	바이너리·설정 파일 변조, 취약한 구버전 의존성	Wazuh FIM, pip-audit
INC-04	네트워크	MockTel 고객 조회 API 대량 조회(IDOR), 기본 관리자 계정	Suricata
INC-05	네트워크	미인가 VM(rogue)의 프로젝트망 접속	arpwatch, Suricata
INC-06	가용성	MockTel로 HTTP 요청 폭주	Suricata threshold, HAProxy, fail2ban

프로젝트 환경

호스트명	IP	역할
attacker	10.10.10.10	시나리오 재현, 부하 생성
lb-01	10.10.10.20	HAProxy, fail2ban, Wazuh 에이전트
web-01	10.10.10.31	MockTel + Nginx + Wazuh 에이전트 (INC-01~03 대상)
web-02	10.10.10.32	MockTel + Nginx + Wazuh 에이전트 (이중화)
monitor	10.10.10.50	Wazuh 올인원(4.14), Suricata, arpwatch
rogue	10.10.10.99	INC-05 때만 켜는 미인가 장비

- 모든 VM: Ubuntu 24.04 Server, 시간대 Asia/Seoul

- 프로젝트망 lab-net: 호스트 전용 네트워크, DHCP 끔, 인터넷 없음 (1~2일차 설치 때만 NAT 연결)
- **모든 패키지는 2일차 NAT 차단 전에 설치한다.** 차단 후에는 apt·pip가 동작하지 않는다. 설치 목록은 프로젝트 문서의 단계별 구축 가이드 1-1단계.
- pip-audit과 gitleaks는 온라인 취약점 정보가 필요하거나 저장소를 대상으로 하므로, 프로젝트망 VM이 아닌 팀원 PC에서 실행한다.
- 스냅샷: 모든 VM에 `baseline` 스냅샷. 시나리오가 꼬이면 되돌린다.

MockTel (취약 웹앱)

가상의 통신사 고객센터. **실제 통신사 이름·로고·도메인을 쓰지 않는다.** Python Flask + SQLite, gunicorn(127.0.0.1:8000) 앞에 Nginx(80). 환경 변수 `SECURE_MODE=0`이면 취약 버전, `1`이면 수정 버전으로 동작한다. `requirements.txt`(수정 버전)와 `requirements-vuln.txt`(취약 버전)의 패키지는 NAT 차단 전에 `/opt/mocktel/wheels/`에 모두 받아 두고, 이후에는 `pip install --no-index --find-links wheels`로만 설치한다. 기능에 새 패키지가 필요하면 차단 전에 avail에 알린다. 뼈대(`core/`)와 기능 파일 틀은 avail이 1~2일차에 만들고, 각 기능의 취약점·수정 코드는 해당 티켓 담당이 채운다.

기능 파일	엔드포인트	취약점 (SECURE_MODE=0)	수정 (SECURE_MODE=1)	티켓
`features/auth.py`	`POST /login`	비밀번호 평문 저장 (`auth_users`)	argon2 해싱	INC-02
`features/config_leak.py`	`GET /debug/config`	DB 비밀번호·API 키 하드코딩·노출 (`LAB-FAKE-` 값)	환경 변수로 분리, 404	INC-02
`features/customer_api.py`	`GET /api/customers/<id>`	IDOR, 횟수 제한 없음	소유자 검사(403), rate limit(429)	INC-04
`features/admin.py`	`POST /admin/login`	기본 계정 admin/admin	기본 계정 제거, 5회 실패 시 잠금	INC-04
`features/search.py`	`GET /search?q=`	제한 없는 무거운 조회	페이지당 20건, 60초 캐시	INC-06
`requirements-vuln.txt`	–	취약점이 알려진 구버전 패키지 1개 (예: `Jinja2==2.10`)	최신 버전	INC-03

모든 응답에 헤더 `X-Served-By: <호스트명>`을 넣어 web-01과 web-02를 구분한다. `/health`는 `ok`를 돌려준다. DB 파일은 `/opt/mocktel/data/mocktel.db`, 앱 코드는 `/opt/mocktel/app/`에 둔다(INC-02·03 탐지 규칙이 이 경로를 본다).

티켓별 상세와 결과 기준

- 티켓별 재현 명령, 탐지 규칙, 기대 결과는 프로젝트 문서의 "티켓 상세 (INC-02~06)" 탭과 메인 탭의 INC-01 상세 진행 계획을 따른다.
- 성공 여부는 메인 탭 "성공 지표" 표로 판단한다: 탐지율 6/6, 탐지 5분 이내, 오탐 2건 이하, SECURE_MODE=1에서 공격 성공 0건, INC-06 오류율 1% 이하·p95 1초 이하.
- 사고 대응 순서와 신고 기한(훈련용)은 "사고 대응 플레이북" 탭을 따른다.

02. 소유권

파일마다 주인은 **한 역할**뿐이다. 주인이 아닌 사람·에이전트는 수정하지 않고, 필요하면 주인에게 요청한다.

역할

역할 키	이름	담당 티켓	담당자
`host`	호스트 탐지	INC-01, INC-02, INC-03	(팀에서 기입)
`net`	네트워크 탐지	INC-04, INC-05	(팀에서 기입)
`avail`	가용성·LB	INC-06, MockTel 뼈대	(팀에서 기입)
`docs`	티켓·문서	전체 티켓 검토, 규약, 보고서	(팀에서 기입)

역할 키는 브랜치 이름 앞부분과 claim 파일에 그대로 쓴다.

경로별 주인

경로	주인	잠금	다른 사람이 할 수 있는 것
`AGENTS.md`, `CLAUDE.md`, `GEMINI.md`, `.cursorrules`, `.github/`	docs	잠금	PR로 제안
`harness/`	docs	잠금	PR로 제안
`docs/` (보고서, 플레이북)	docs	–	PR로 제안
`app/mocktel/core/` (앱 생성, DB 연결, 공통 레이아웃)	avail	잠금	수정 금지
`app/mocktel/features/auth.py`, `config_leak.py`	host	–	수정 금지
`app/mocktel/features/customer_api.py`, `admin.py`	net	–	수정 금지
`app/mocktel/features/search.py`	avail	–	수정 금지
`app/requirements.txt`, `app/requirements-vuln.txt`	avail	–	INC-03은 host가 avail에 요청
`detection/wazuh/`	host	–	수정 금지
`detection/suricata/`	net	–	INC-06 규칙은 avail이 net에 요청
`infra/netplan/`, `infra/ufw/`, `infra/hosts`	docs	잠금	수정 금지
`infra/haproxy/`, `infra/fail2ban/`, `infra/nginx/`	avail	–	수정 금지
`scenarios/INC-01/ ~ INC-03/`	host	–	수정 금지
`scenarios/INC-04/`, `INC-05/`	net	–	수정 금지
`scenarios/INC-06/` (k6 `load.js` 포함)	avail	–	수정 금지
`tests/postman/` (MockTel 보안 테스트 컬렉션, secure0·secure1 환경)	net	–	다른 역할은 테스트 추가를 net에 요청

경로	주인	잠금	다른 사람이 할 수 있는 것
`tickets/INC-0X/ticket.md`, `evidence/`	해당 티켓 담당	–	수정 금지
`tickets/INC-0X/worklog.md`	해당 티켓 담당	–	**덧붙이기만** 허용
`.claims/<티켓>.claim`	claim을 만든 사람	–	수정·삭제 금지

기능 추가 규칙 (MockTel)

- 1~2일차에 avail이 `core/`와 모든 기능 파일의 빈 틀을 만든다. 틀이 main에 머지된 순간부터 각 기능 파일의 소유권은 위 표의 주인에게 넘어간다.
- 새 기능은 `features/` 아래 **새 파일**로 만든다. 기존 파일에 끼워 넣지 않는다.
- 기능 파일은 `core/`가 제공하는 `register(app)` 방식으로만 등록한다. 등록 목록 파일(`core/registry.py`)은 avail만 고친다. 다른 역할은 "이 파일을 등록해 달라"고 avail에 요청한다.
- DB 테이블은 기능별 접두사를 쓴다. `auth_users`, `cust_customers`, `admin_accounts`, `search_logs`. 같은 테이블을 두 기능이 같이 쓰지 않는다.

잠금 파일을 바꿔야 할 때

1. 바꿀 내용과 이유를 PR 설명이나 메시지로 docs에 보낸다.
2. docs가 직접 바꾸거나 승인한다.
3. 승인 전에는 그 변경에 기대는 작업을 진행하지 않는다.

03. 번호 등록부

이 표에 없는 번호는 쓰지 않는다. 새 번호가 필요하면 docs에 요청하고, 등록된 뒤에 쓴다. 대역 안에서 실제로 쓴 번호는 아래 "사용 기록"에 줄을 추가한다(해당 티켓 담당만, 덧붙이기만).

Wazuh 규칙 ID

대역	티켓	파일
100100–100109	공통(docs)	`detection/wazuh/rules/common_rules.xml`
100110–100119	INC-01	`detection/wazuh/rules/inc01_rules.xml`
100120–100129	INC-02	`detection/wazuh/rules/inc02_rules.xml`
100130–100139	INC-03	`detection/wazuh/rules/inc03_rules.xml`
100140–100149	INC-04	`detection/wazuh/rules/inc04_rules.xml`
100150–100159	INC-05	`detection/wazuh/rules/inc05_rules.xml`
100160–100169	INC-06	`detection/wazuh/rules/inc06_rules.xml`

- 규칙 파일은 monitor의 `/var/ossec/etc/rules/`에 그대로 복사한다. `local_rules.xml`은 쓰지 않는다.
- 각 파일의 `<group name="...">`은 `inc0X,`로 시작한다. 예: `<group name="inc01,">`

Wazuh 에이전트 설정 조각

티켓	파일	alias 접두사
INC-01	`detection/wazuh/agent/inc01_localfile.xml`	`inc01_`
INC-02	`detection/wazuh/agent/inc02_localfile.xml`	`inc02_`
INC-03	`detection/wazuh/agent/inc03_syscheck.xml`	–

조각을 실제 `ossec.conf`에 합치는 일은 host만 한다.

Suricata SID

대역	티켓	파일
1000001–1000099	공통(docs)	`detection/suricata/common.rules`
1000400–1000499	INC-04	`detection/suricata/inc04.rules`
1000500–1000599	INC-05	`detection/suricata/inc05.rules`
1000600–1000699	INC-06	`detection/suricata/inc06.rules`

- 티켓 규칙의 `msg`는 `"INC-0X: "`로, 공통 규칙(1000001–1000099)의 `msg`는 `"COMMON: "`으로 시작한다.
- monitor에서는 이 파일들을 합쳐 `/etc/suricata/rules/local.rules`로 배포한다(net만).

호스트명·IP (변경 금지)

호스트명	IP
attacker	10.10.10.10
lb-01	10.10.10.20
web-01	10.10.10.31
web-02	10.10.10.32
monitor	10.10.10.50
rogue	10.10.10.99
(호스트 PC)	10.10.10.1

Suricata 주소 변수 `LAB_KNOWN`(INC-05용)은 위 표의 rogue를 뺀 6개 주소로만 구성한다. 호스트가 바뀌면 docs가 이 표와 함께 고친다.

포트

포트	위치	용도	주인
22/tcp	전체	SSH (lab-net에서만)	docs
80/tcp	lb-01, web-01, web-02	HTTP	avail
8000/tcp	web-01, web-02 (127.0.0.1)	gunicorn	avail
443/tcp	monitor	Wazuh 대시보드	host
1514, 1515/tcp	monitor	Wazuh 에이전트	host
55000/tcp	monitor	Wazuh API	host
4444/tcp, 9999/udp	web-01	INC-01 재현 전용	host

스냅샷 이름

이름	시점
`baseline`	단계별 구축 가이드 8단계 완료 직후
`before-INC-0X`	해당 티켓 재현 직전(필요할 때만)

사용 기록

날짜	티켓	번호	설명
(예) 2026-10-20	INC-01	100110	삭제되었거나 임시 경로에서 실행 중인 프로세스
(예) 2026-10-20	INC-01	100111	패킷 소켓 목록 변경

04. 작업 절차

사람과 LLM 에이전트 모두 이 순서를 그대로 따른다. 단계를 건너뛰지 않는다.

1. 시작 전 확인

```
git checkout main && git pull
ls .claims/
```

- 내 티켓의 claim 파일이 이미 있고 주인이 다르면 **멈춘다.**
- 내 작업이 수정할 경로를 다른 claim이 이미 적어 두었으면 **멈춘다.**

2. 선점 (claim)

`.claims/INC-0X.claim` 파일을 만든다. 하위 작업이면 `INC-0X-<설명>.claim`. 형식은 `harness/06-templates.md`의 claim 템플릿을 따른다.

claim은 **main에서 직접** 커밋·푸시한다. 이것이 R5의 유일한 예외이며, 이 커밋에는 `.claims/` 파일 외에 다른 변경을 넣지 않는다.

```
git add .claims/INC-01.claim
git commit -m "[INC-01] claim: 패킷 소켓 탐지 규칙 작업 시작"
git push
```

- 파일 이름이 작업마다 다르므로 claim끼리는 충돌하지 않는다.
- 푸시가 거절되면(다른 사람이 먼저 같은 이름으로 올림) `git pull` 후 1단계부터 다시 한다.

3. 브랜치

```
<역할>/<티켓>-<짧은-영문-설명>
```

예: `host/INC-01-packet-socket-rule`, `net/INC-04-idor-threshold`, `avail/INC-06-haproxy-failover`

```
git checkout -b host/INC-01-packet-socket-rule
```

4. 작업

- claim의 `paths`에 적은 경로만 수정한다.
- 범위가 늘면 먼저 claim을 고쳐 커밋하고 나서 작업한다.
- 번호(규칙 ID, SID, 포트)는 `03-id-registry.md`의 내 대역에서만 고른다.

5. 커밋

```
[INC-0X] <종류>: <한국어 한 줄 설명>

<필요하면 본문>

Agent: <도구 이름, 사람이면 생략>
```

종류	언제
`claim`	선점, 선점 해제
`detect`	탐지 규칙, 에이전트 설정
`scenario`	재현·정리 스크립트
`app`	MockTel 코드
`infra`	HAProxy, Nginx, fail2ban 등 설정
`ticket`	티켓, worklog, 증거
`docs`	규약, 보고서, 플레이북

한 커밋에는 한 종류만 담는다. 저장소 초기 설정처럼 티켓이 없는 작업은 `[INC-0X]` 대신 `[SETUP]`을 쓴다(docs만). 예: `[SETUP] docs: 하네스 추가`

6. worklog 기록

`tickets/INC-0X/worklog.md` **끝에** 항목을 추가한다. 기존 줄은 고치지 않는다. 형식은 템플릿을 따른다.

7. PR

```
git fetch origin && git rebase origin/main
git push -u origin host/INC-01-packet-socket-rule
```

- PR 제목은 첫 커밋 제목과 같은 형식을 쓴다.
- PR 본문은 템플릿을 따른다.
- rebase 중 **내 소유가 아닌 파일**에서 충돌이 나면 직접 해결하지 않는다. `git rebase --abort` 후 그 파일 주인에게 알린다.
- 머지는 사람만 한다. 최소 1명(다른 역할)의 리뷰가 필요하다.

8. 종료

PR이 머지된 뒤 claim을 main에서 직접 지운다(R5 예외, `.claims/` 변경만 담는다).

```
git checkout main && git pull
git rm .claims/INC-01.claim
git commit -m "[INC-01] claim: 패킷 소켓 탐지 규칙 작업 완료"
git push
```

그리고 프로젝트 문서의 로드맵 상태 칸을 갱신한다.

충돌이 났을 때 판단표

상황	할 일
같은 티켓을 두 사람이 claim	먼저 푸시된 쪽이 가진다. 늦은 쪽은 claim을 지우고 다른 작업을 받는다.
내 파일인데 rebase 충돌	직접 해결하고 worklog에 남긴다.
남의 파일에서 rebase 충돌	중단하고 주인에게 넘긴다.
같은 규칙 ID를 두 곳에서 씀	등록부 대역을 확인한다. 대역 밖을 쓴 쪽이 고친다.
잠긴 파일을 고쳐야 함	docs에 요청하고, 승인 전에는 기대는 작업을 멈춘다.
에이전트가 범위 밖 수정을 제안	적용하지 말고 제안 내용만 worklog에 남긴다.

05. 안전 규칙

이 프로젝트는 방어 목적이며, 우리 팀이 소유한, 외부와 분리된 프로젝트망 안에서만 진행한다.

대상

- 허용: `10.10.10.0/24` 안의 우리 VM(attacker, lb-01, web-01, web-02, monitor, rogue)
- 금지: 프로젝트망 밖의 모든 주소. 실제 통신사·기업·공개 서비스, 인터넷 주소, 학교망, 팀원 PC의 다른 네트워크
- 스크립트의 대상 주소는 하드코딩하지 말고 `infra/hosts`의 호스트명을 쓴다. 대상이 `10.10.10.` 대역이 아니면 실행을 멈추는 검사를 넣는다.

```
# 모든 scenarios/*.sh 맨 위에 넣는다
TARGET_IP=$(getent hosts "$TARGET" | awk '{print $1}')
case "$TARGET_IP" in 10.10.10.*) ;; *) echo "프로젝트망 밖 대상: $TARGET_IP"; exit 1;; esac
```

공격 재현

- 실제 악성코드, 익스플로잇 키트, 공개 PoC 바이너리를 내려받거나 만들지 않는다.
- 기법은 기본 명령으로만 흉내 낸다: `nc`, `sleep`, `tcpdump`, `curl`, `k6`, `nmap`.
- 재현에 쓴 프로세스와 파일은 `scenarios/INC-0X/cleanup.sh`로 반드시 정리한다.
- 부하 테스트는 NAT를 끊은 상태에서만 한다.

비밀값

- MockTel의 하드코딩 비밀값, 테스트 계정 비밀번호는 `LAB-FAKE-`로 시작하는 가짜 값만 쓴다.
- Wazuh admin 비밀번호, `wazuh-install-files.tar`, SSH 키는 저장소·채팅방·티켓에 넣지 않는다. 팀 비밀번호 관리자에만 둔다.
- 커밋 전에 `gitleaks detect --source .`를 돌린다. 단, `LAB-FAKE-` 값은 허용 목록(`.gitleaks.toml`)에 등록한다.

MockTel

- 실제 통신사 이름·로고·도메인·디자인을 쓰지 않는다.
- 실제 사람의 개인정보를 넣지 않는다. 고객 데이터는 가짜로 생성한다.
- MockTel은 프로젝트망 밖에 배포하지 않는다(클라우드, 공개 저장소의 데모 링크 포함).

증거 보존

- 프로세스를 죽이거나 VM을 되돌리기 **전에** 증거부터 수집한다.
- 증거는 `tickets/INC-0X/evidence/`에 두고, 파일마다 수집 시각과 수집 명령을 같은 이름의 `.txt`로 남긴다.
- 실행 파일 복사본(`/proc/<PID>/exe` 등)은 실행 권한을 빼고(`chmod 000`) 압축해서 보관한다.

사고 신고 연습

실제 신고는 하지 않는다. 플레이북에는 "개인정보 유출 시 72시간 이내 신고" 같은 법적 기한을 적고, 테이블탑 훈련에서 그 기한 안에 누가 무엇을 하는지만 연습한다.

06. 템플릿

claim (`.claims/INC-0X.claim`)

```
ticket: INC-01
role: host
owner: 홍길동
agent: claude-code        # 사람이 직접 하면 none
started: 2026-10-20 10:00
branch: host/INC-01-packet-socket-rule
paths:
  - detection/wazuh/rules/inc01_rules.xml
  - detection/wazuh/agent/inc01_localfile.xml
  - scenarios/INC-01/
  - tickets/INC-01/
summary: 패킷 소켓, 삭제된 실행 파일 탐지 규칙 추가
```

worklog 항목 (`tickets/INC-0X/worklog.md` 끝에 추가)

```
## 2026-10-20 14:10 | owner: 홍길동 | agent: claude-code
- 바꾼 것: detection/wazuh/rules/inc01_rules.xml (규칙 100110, 100111 추가)
- 확인한 것: web-01에서 B·C 재현 → 대시보드에서 100110, 100111 알림 확인
- 못 한 것: 주기를 60초로 줄인 뒤 CPU 사용량은 측정 못 함
```

인시던트 티켓 (`tickets/INC-0X/ticket.md`)

```
티켓 ID     : INC-0X
제목        :
근거 사례   :
탐지 시각   :
탐지 경로   : (NIDS / HIDS / 서비스 로그, 규칙 ID)
심각도      : (치명 / 높음 / 중간 / 낮음)
영향 자산   :
ATT&CK      :
타임라인    : 탐지 → 분류 → 격리 → 증거 보존 → 제거 → 복구
증거 보존   : (무엇을, 어디에 저장했는가)
조치 내용   :
결과        : (보강 전후 비교, SECURE_MODE 0/1 비교)
상태        : (열림 / 대응 중 / 종결)
교훈        :
```

PR 본문

```
## 티켓
INC-0X

## 바꾼 것
-

## 확인한 것 (실행한 명령과 결과)
-

## 못 한 것 / 확인 못 한 것
-

## 체크리스트
- [ ] claim의 paths 밖 파일을 고치지 않았다
- [ ] 번호는 03-id-registry.md 대역 안에서만 썼고, 사용 기록에 추가했다
- [ ] gitleaks 통과 (LAB-FAKE- 제외)
- [ ] 대상 주소는 모두 10.10.10.0/24
- [ ] worklog에 항목을 추가했다
```

시나리오 스크립트 머리말 (scenarios/INC-0X/reproduce.sh)

```
#!/usr/bin/env bash
# INC-0X 재현 스크립트 — 프로젝트망 전용
# 근거 사례:
# 실행 위치: (VM 이름)
# 정리: ./cleanup.sh
set -euo pipefail
TARGET="${TARGET:-lb-01}"
TARGET_IP=$(getent hosts "$TARGET" | awk '{print $1}')
case "$TARGET_IP" in 10.10.10.*) ;; *) echo "프로젝트망 밖 대상: $TARGET_IP"; exit 1;; esac
```

Wazuh 규칙 파일 머리말 (detection/wazuh/rules/inc0X_rules.xml)

```
<!-- INC-0X | 대역 1001X0-1001X9 | 주인: host -->
<group name="inc0X,">
</group>
```

Suricata 규칙 형식 (detection/suricata/inc0X.rules)

```
# INC-0X | SID 10000X00-10000X99 | 주인: net
alert http any any -> $HOME_NET 80 (msg:"INC-0X: <설명>"; ...; sid:10000X01; rev:1;)
```